

Actividad 5 — Redis: base clave-valor y patrón de caché

Datos generales

- **Duración:** 50 minutos
- **Tipo:** Individual
- **Herramienta de IA:** Libre
- **Requisitos:** Docker funcionando

Presupuesto de tiempo

Paso	Tiempo
Leer el marco teórico	5 min
Levantar Redis con Docker	5 min
Tipos de datos: strings, hashes, listas, sets	15 min
TTL y expiración	8 min
Patrón de caché	10 min
Verificación y entrega	7 min

Objetivos

1. Comprender el modelo de datos **clave-valor** y su importancia en memoria.
2. Usar los principales tipos de datos de Redis: STRING, HASH, LIST, SET, SORTED SET.
3. Manejar **expiración (TTL)** de claves.
4. Implementar el **patrón de caché** (cache-aside).

Marco teórico

¿Qué es Redis?

Redis (REmote DIctionary Server) es una base de datos NoSQL **en memoria**, open source. Guarda **pares clave-valor** a velocidad de RAM. Es la base más rápida del curso, pero a cambio los datos dependen de la memoria.

Estructura

Clave (string) → Valor (uno de los tipos de datos)

Tipos de datos

Tipo	Ejemplo de comando	Uso típico
STRING	SET usuario:1 "..."	caché simple, contadores
HASH	HSET usuario:1 nombre Ana	objetos con campos
LIST	LPUSE cola "job"	colas, historial
SET	SADD amigos:1 "2"	membresía, conjuntos
ZSET	ZADD ranking 100 "ana"	rankings ordenados

TTL (Time To Live)

Cada clave puede tener un **tiempo de vida**. Al vencer, Redis la borra sola. Es la base del patrón caché.

Patrón cache-aside (la única caché no escrita en el código)

1. La app pide el dato → Redis (GET)
2. ¿Existe? Sí → devuelve el dato (hit)
3. ¿No existe? → la app lo lee de MongoDB (miss)
4. La app lo guarda en Redis con TTL → lo devuelve

Esto reduce lecturas a la base principal. El TTL evita que quede **información vieja** para siempre.

Paso a paso

Paso 1 — Levantar Redis

```
docker run -d --name redis-clase \
  -p 6379:6379 \
  redis:7
```

Entrá a la consola de Redis:

```
docker exec -it redis-clase redis-cli
```

Deberías ver el prompt 127.0.0.1:6379>.

Paso 2 — Strings y contadores

```
SET mensaje "hola nosql"
GET mensaje
```

Contadores

```
SET visitas 0
INCR visitas
INCR visitas
```

```
INCRBY visitas 5
GET visitas
```

Paso 3 — Hashes (objetos)

```
HSET usuario:1 nombre "Ana" ciudad "Montevideo" edad 30
HGET usuario:1 nombre
HGETALL usuario:1
HINCRBY usuario:1 edad 1
```

Paso 4 — Listas (colas)

```
LPUSH cola:tareas "procesar-venta-1"
LPUSH cola:tareas "procesar-venta-2"
LRANGE cola:tareas 0 -1
RPOP cola:tareas
LPUSH cola:tareas "procesar-venta-3"
LLEN cola:tareas
```

Paso 5 — Sets (membresía)

```
SADD online "ana"
SADD online "bruno"
SADD online "ana"           # no agrega duplicados
SMEMBERS online
SISMEMBER online "carla"
```

Paso 6 — Sorted Sets (rankings)

```
ZADD ranking 100 "ana"
ZADD ranking 250 "bruno"
ZADD ranking 180 "carla"
ZRANGE ranking 0 -1 WITHSCORES
ZREVRANGE ranking 0 2      # Los 3 mejores (de mayor a menor)
```

Paso 7 — Expiración (TTL)

```
SET token:sesion "abc123"
EXPIRE token:sesion 10
TTL token:sesion          # cuántos segundos quedan
GET token:sesion
```

```
# Esperá 10 segundos y volvé a consultar
GET token:sesion          # → (nil), la clave venció
```

```
# Forma corta: SET con tiempo de vida
SET promocion "20% off" EX 15
TTL promocion
```

(nil) en Redis significa que la clave no existe (venció).

Paso 8 — Patrón cache-aside en la terminal

Simulamos la caché de un perfil que en "MongoDB" tardaría 1 segundo:

1. La app pregunta a Redis

GET cache:perfil:1

→ (nil) → miss de caché

2. (En un caso real) se Leería de MongoDB aquí

3. La app guarda en Redis con TTL

SET cache:perfil:1 '{"nombre":"Ana","ciudad":"Montevideo"}' EX 30

4. La próxima consulta es un hit

GET cache:perfil:1

5. El TTL garantiza que en 30 seg la caché se refresque

TTL cache:perfil:1

Paso 9 — Ver el estado de la base

DBSIZE

KEYS * *# ojo: no usar en producción con muchas claves*

FLUSHALL *# borra todo (solo en prácticas)*

Paso 10 — Preguntar a la IA

¿Cuál es la diferencia entre Redis y Memcached? ¿Cuándo conviene cada uno?
¿Qué pasa con la consistencia si la app escribe primero en MongoDB y después en Redis, o al revés?

Mostrame un ejemplo real de cómo Python o Node.js usarían Redis como caché con TTL.

Verificación de resultados

- ☐ El contador del paso 2 llegó a 7.
- ☐ El hash usuario:1 muestra todos sus campos con HGETALL.
- ☐ Una clave con EXPIRE 10 desaparece después de 10 segundos.
- ☐ El patrón cache-aside muestra primero (nil) y después el valor guardado.
- ☐ ZREVRANGE ordena el ranking correctamente.

Criterios de evaluación

Criterio	Puntos
Contenedor Redis levantado	10
Strings + hashes correctos	20
Listas + sets + sorted sets	25
TTL y expiración comprendidos	20

Criterio	Puntos
Patrón cache-aside implementado	25

Entregable

- Archivo `redis-kv.md` con los comandos y resultados de cada paso.
- Respuesta de la IA (captura) sobre el problema de consistencia de la caché.
- Ejemplo de la pregunta 3 respondida: cómo usaría tu app la caché de Redis.

Seguridad (adelanto)

Redis se lanza **sin contraseña y sin red restringida**: si publicás el puerto 6379, cualquiera puede ejecutar `FLUSHALL` y borrar todo. En la próxima actividad vas a protegerlo con `requirepass`, **ACLs** y renombramiento de comandos peligrosos.